Chess AI — Technical Report

1. Objective

The goal of this project was not to build a chess engine. I wanted to build a system that plays chess in my style.

That creates a different problem. The strongest move is not necessarily the move I would have played, and occasionally a mistake is actually part of the behavior I am trying to reproduce.

The final system therefore combines three things:

- An opening book based on my games
- Stockfish for generating candidate moves
- A neural network that chooses between those candidates based on my historical decisions

The opening book handles the part of my play that is relatively consistent. Once outside the book, the neural network takes over.

---

2. The Dataset

I used roughly 5,000 of my games, collected over around five years.

The biggest problem with this dataset is that it contains several versions of me.

My playing strength changed substantially during those five years. My older games reflect a much weaker player than my recent games, so treating every game equally would not really produce a model of my current style.

I therefore wanted the newer games to have more influence during training. This also meant that blindly shuffling everything together was not ideal.

This is an unusual problem for a dataset: some of the "noise" is actually me, just an older version of me.

Another problem is that positions within the same game are highly correlated. A random position-level train/validation split would therefore leak information between the two sets. I split by game, not by position.

Even then, the effective dataset is much smaller than the number of positions suggests because many positions come from the same games.

---

3. Defining "Style"

There is no obvious ground-truth value for chess style.

I ended up treating style as a prediction problem:

Given the same candidate moves, how highly does the model rank the move I actually played?

The main metrics were therefore Top-K accuracy and ranking-based metrics.

This is also why Stockfish is useful without becoming the actual target. Stockfish tells me which moves are reasonable candidates, while my games tell the network which of those moves I tend to prefer.

The final evaluation on 10,055 held-out positions gave:

Metric	Result
Top-1	4077 / 10055 (40.55%)
Mean P(actual move), candidate exists	0.3112
Median P(actual move)	0.2128
Mean Reciprocal Rank	0.6137

The Top-1 number should not be interpreted as chess-playing accuracy. It means that my actual move was the model's first choice among the available candidates 40.55% of the time.

---

4. Candidate Generation

I originally considered making the network predict directly from all possible chess moves. Instead, I used Stockfish to generate up to 13 candidate moves and trained the network to rank them.

This makes the learning problem considerably smaller, but it creates an important limitation: the model cannot choose a move that Stockfish did not give it.

In the evaluation, my actual move was present in the candidate set in about 92% of the positions.

The remaining positions are therefore not purely neural-network failures. The desired answer was not available in the input.

This also led to another implementation problem: the number of candidates is variable. Some positions have one meaningful candidate while others have many.

The network expects a fixed-size input, so candidate lists were padded to 13 and an explicit padding feature was added. Mate candidates were handled separately from ordinary evaluations.

---

5. Move Representation

A move is represented using more than its UCI notation.

The network receives:

- From/to squares
- Engine evaluation
- Capture and check indicators
- Moving piece
- Captured piece
- Promotion type
- Padding and mate indicators

The board itself is represented using 12 piece planes.

I also changed the board representation to be relative to the side playing rather than treating White and Black as completely separate

cases. This makes the same type of position useful for both sides and allows more effective augmentation.

---

6. Data Augmentation

The relative representation made it possible to horizontally flip positions and their moves.

This gives another effective doubling of the dataset without collecting additional games.

There are exceptions to perfect symmetry, particularly around castling and other state-dependent information, so the transformation cannot simply be treated as a universal chess symmetry.

It was nevertheless a useful source of additional training data.

---

7. The Opening Book

The opening book was added for two reasons.

First, my opening choices are more consistent than many of my later decisions. Second, allowing the neural network to make arbitrary opening moves can push it into positions that are poorly represented in its training data.

Using the book keeps the early game closer to positions I actually played and leaves the neural network to deal mainly with the more interesting part of the problem.

The book itself introduced some problems. Chess positions can transpose into one another through different move sequences, while a sequence-based book does not necessarily recognize them as the same position.

A more complete implementation would index the book by position rather than relying primarily on the move sequence.

---

8. The Problem With Blunders

One of the more interesting problems was that blunders are rare.

If the objective is to imitate my style, occasionally making the same kind of mistake is useful. But the training data contains very few examples of any particular mistake.

The network therefore has a natural tendency to become a better chess player than the person it is supposed to imitate.

That is not necessarily an improvement for this project.

The objective is not simply to maximize playing strength. It is to reproduce the distribution of decisions I tend to make.

---

9. Engine Evaluation as a Feature

Stockfish evaluations are useful features, but they also provide the network with a shortcut.

If the evaluation is always available, the easiest strategy can become "choose the move with the best evaluation" rather than actually learning my preferences.

I normalized the engine evaluations before feeding them to the network and experimented with evaluation dropout so that the model could not rely on the engine score for every decision.

This was particularly important because the whole point of the network is to learn something beyond Stockfish.

---

10. Probability and Evaluation

The model produces a softmax distribution over the candidate moves.

I initially considered interpreting these values directly as probabilities of me playing each move. In practice, that interpretation is too strong.

Most chess positions occur only once in the dataset, so there is rarely enough repeated evidence to say that a move has a true probability of, for example, 30%.

The distribution is more useful as a measure of relative model preference.

This is why Top-K accuracy and ranking metrics are more meaningful than treating the softmax output as calibrated behavioral probabilities.

---

11. Repetition and History

The current model mainly sees the current board and candidate moves.

That is enough for most move decisions, but not for rules that depend on previous positions.

Threefold repetition is an obvious example. The same board can have different repetition histories depending on how it was reached.

A more complete model would therefore need some representation of move history.

This is one of the places where the problem stops being a simple board-to-move prediction task.

---

12. Overfitting

The dataset is not particularly large once game correlation is taken into account.

The network could therefore overfit relatively quickly, especially because the input contains detailed information about individual candidate moves.

Weight decay was necessary to keep the model from fitting the training data too aggressively.

The game-level validation split was equally important here, otherwise the validation results could look much better than the actual generalization performance.

---

13. Preprocessing

A significant amount of work ended up going into preprocessing rather than the model itself.

The original samples had inconsistent candidate lengths, missing evaluations, padding issues and malformed entries.

The preprocessing pipeline eventually handled:

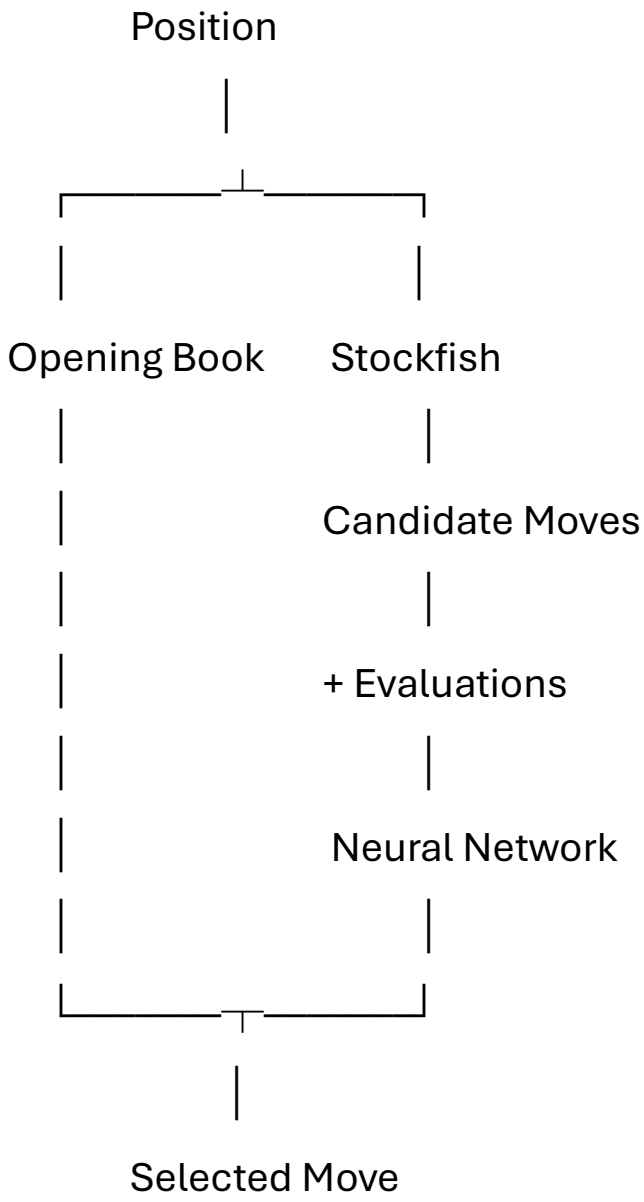
- Evaluation normalization
- Candidate padding
- Mate indicators
- Dummy moves
- Board augmentation
- Candidate transformations
- Game-level dataset preparation

This was necessary before the training results became meaningful.

---

14. What the Final System Represents

The final architecture can be summarized as:



The opening book provides my established opening preferences.

Stockfish provides a reasonable search space.

The neural network provides the actual imitation component.

---

15. Limitations and Next Steps

There are several things I would change in a second version.

The biggest limitation is still the dataset. Five thousand games is enough to make the experiment interesting, but it is small compared with the amount of data available to commercial style-replication systems, and my games do not represent one completely consistent playing style.

I would also like to:

- Handle opening-book transpositions properly
- Incorporate move history
- Improve the representation of variable candidate sets
- Investigate predicting moves outside the Stockfish candidate set
- Weight games according to the evolution of my playing strength more systematically
- Explore whether a model can learn when to deliberately reproduce human mistakes

There is also a more fundamental question that remains unresolved:

How should human chess style actually be measured?

Top-K prediction of historical moves is a practical approximation, but it is not a complete definition of style. A stronger evaluation would ideally compare games produced by the model against my games using several behavioral characteristics rather than one prediction metric.

Conclusion

This project started as "train a neural network on my chess games."

It turned out to be much more of a data and problem-definition exercise.

The hardest parts were not getting a network to output a move. They were deciding what the network should learn, what information it should be allowed to use, how to prevent Stockfish from solving the problem for it, and how to measure whether the resulting decisions actually resemble mine.

The final system is therefore deliberately not the strongest chess player I could build.

It is an attempt to build a chess player that makes decisions the way I do.